

TRABAJO PRÁCTICO INTEGRAL: FOODSTORE

Asignatura: Programación III

Institución: Universidad Tecnológica Nacional (UTN)

Estudiante: Francisco López (DNI: 39.327.419)

1. Introducción y Arquitectura del Proyecto

El presente documento detalla los aspectos técnicos y de diseño aplicados en el desarrollo del Trabajo Práctico Integral (TPI) para la plataforma FoodStore.

Con el fin de garantizar un software mantenible, escalable y robusto, el proyecto se estructuró bajo una arquitectura modular compuesta por dos sistemas desacoplados que operan bajo reglas de negocio idénticas en paralelo:

-Módulo Backend (Java 17): Capa encargada de la lógica transaccional, el control secuencial de los estados y la persistencia física en base de datos.

-Módulo Frontend (React & TypeScript): Interfaz SPA (Single Page Application) modularizada para emular de forma interactiva y reactiva la experiencia del cliente y del administrador.

2. Estructura y Tecnologías del Frontend (VS Code)

La interfaz de usuario fue desarrollada utilizando React y Vite como empaquetador ágil, estilizada con Tailwind CSS y estructurada estrictamente bajo TypeScript para evitar inconsistencias de tipos. El código se organiza en tres pilares principales dentro de la carpeta src:

-pages: Agrupa las vistas autónomas de la aplicación, separando las pantallas del administrador (admin), los formularios de autenticación (auth), el flujo general del cliente (client), y la góndola comercial de pedidos (store).

-types: Define las interfaces de tipado estricto (como product.ts, categoria.ts y Rol.ts) para representar fielmente la estructura de datos que maneja el negocio.

-utils: Centraliza funciones utilitarias y la persistencia local de la sesión de usuario activa (localStorage.ts).

La autenticación simula el ingreso seguro validando de forma local las credenciales del operador contra el archivo de configuración estático usuarios.json provisto por la cátedra.

3. Desarrollo del Backend y Persistencia (JPA/IntelliJ)

La persistencia del Backend está implementada mediante JPA (Jakarta Persistence API) y Hibernate como proveedor de ORM (Mapeo Objeto-Relacional), utilizando un motor de base de datos SQL H2 embebido en archivo para asegurar la conservación de los datos.

3.1 Mapeo de Entidades Clave

-Usuario: Define los datos del cliente y administrador, persistiendo su perfil mediante el uso de la anotación `@Enumerated(EnumType.STRING)` sobre el enum Rol.

-Categoria y Producto: Incorporan atributos de baja lógica para desactivar elementos del catálogo sin comprometer la integridad histórica de pedidos ya facturados.

-Pedido: Representa la cabecera transaccional de compra. Implementa la interfaz Calculable y se vincula mediante una relación @OneToMany en cascada estricta (CascadeType.ALL, orphanRemoval = true) con la entidad DetallePedido.

3.2 Repositorio Base Genérico

Para evitar la duplicidad de lógica en las transacciones estándar de persistencia, estructuré una interfaz genérica parametrizada de la cual heredan los repositorios específicos (PedidoRepository, ProductoRepository, etc.):

Java:

```
public interface BaseRepository<T, ID> {  
    T findById(ID id);  
    List<T> findAll();  
    T save(T entity);  
    void delete(T entity);  
}
```

4. Lógica de Negocio y Casos de Uso (HU)

4.1 Transaccionalidad y Descuento de Stock (HU-12 y HU-13)

Al momento de confirmar un pedido, la lógica de negocio del Backend procesa la operación dentro de un bloque transaccional:

- 1- Se verifica la existencia y estado activo de las entidades en la base de datos H2.
- 2- Se calculan los subtotales de cada ítem y el monto total final del pedido de manera dinámica.
- 3- Se actualiza en caliente el stock físico de la entidad Producto mediante un comando UPDATE en SQL. Si la cantidad solicitada supera al stock físico existente, la transacción se interrumpe para preservar la integridad del inventario.

4.2 Control Secuencial del Ciclo de Vida (HU-14)

El ciclo de vida de los pedidos se controla mediante transiciones estrictamente ordenadas y validadas por código:

PENDIENTE -> CONFIRMADO -> TERMINADO

El sistema impide de forma nativa saltos de estado inválidos, garantizando la consistencia del flujo operativo.

5. Consultas Complejas y Reportes Analíticos (JPQL)

Para brindar herramientas analíticas a la toma de decisiones, se programaron consultas avanzadas utilizando JPQL (Java Persistence Query Language):

-Reporte de Facturación Agrupada (HU-16): Obtiene los consumos históricos totales de cada usuario aplicando funciones de agregación y agrupamiento relacional:

SQL:

```
SELECT p.usuario.nombre, SUM(p.total)  
FROM Pedido p  
GROUP BY p.usuario.nombre
```

-Reporte de Clientes Premium (HU-17): Filtra y aísla a los compradores cuyos consumos individuales superan el promedio general de facturación de la tienda mediante una subconsulta dinámica con AVG:

SQL:

```
SELECT p.usuario  
FROM Pedido p  
WHERE p.total > (SELECT AVG(p2.total) FROM Pedido p2)
```